Foundation of Blockchain Technology (IS501)
Post-Reading Material – Lecture 8

SHA-256

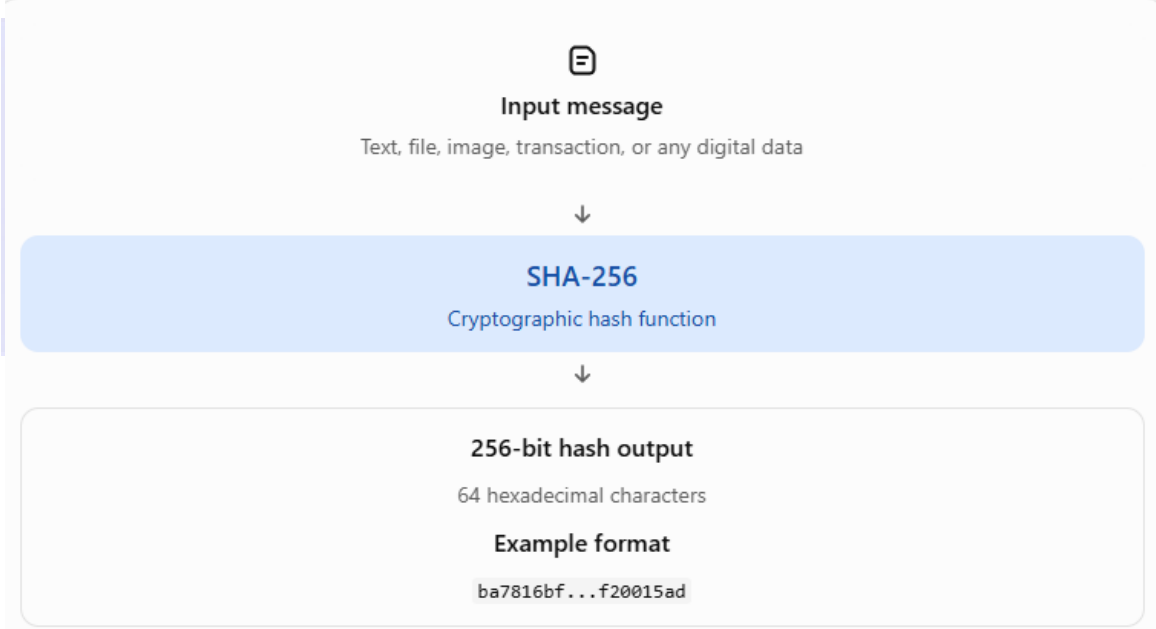
SHA-256 is one of the most widely used cryptographic hash functions. It is used in blockchain, digital signatures, password-storage systems, file-integrity checking, and cybersecurity.

Let us understand it from the basics, then explore its mathematics, internal working, and a complete example.

What is SHA-256?

SHA-256 stands for Secure Hash Algorithm 256-bit. It belongs to the SHA-2 family of cryptographic hash functions.

It takes input data of almost any length and produces a fixed-length output of 256 bits.



For example, the input message `abc` produces this SHA-256 hash:

Input

```
abc
```

SHA-256 output

ba7816bf8f01cfea414140de5dae2223
b00361a396177a9cb410ff61f20015ad

Notice that the output is always 256 bits, regardless of whether the input is `abc`, a full book, or a large video file.

Why is it called a 256-bit hash?

Each SHA-256 output contains exactly 256 binary digits (bits).

A hexadecimal digit represents four bits. Therefore,

$$\frac{256}{4} = 64$$

So, every SHA-256 hash is represented by 64 hexadecimal characters.

For example:

- Binary: `1010`
- Hexadecimal: `A`
- Four hexadecimal digits: 16 bits
- 64 hexadecimal digits: 256 bits

The number of possible 256-bit outputs is

$$2^{256} \approx 1.16 \times 10^{77}$$

That is an extraordinarily large number of possible hash values.

Important: A 256-bit hash does not mean the input is limited to 256 bits. It means the output is 256 bits long.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Five important properties of SHA-256:

1. Fixed-length output

Any input length produces a 256-bit hash.

2. Deterministic

The same input always produces the same hash, assuming the same byte representation.

3. Avalanche effect

A tiny change in the input should change roughly half the output bits, on average.

4. One-way design

Given a hash, it should be computationally infeasible to recover an arbitrary original message from it.

5. Collision resistance

It should be computationally infeasible to deliberately find two different inputs that produce the same hash.

Understanding the avalanche effect

Consider these two inputs:

- `Hello`
- `hello`

They differ by only one character's capitalization. However, their SHA-256 hashes look completely unrelated.

This is intentional. An attacker should not be able to infer how similar two inputs are simply by comparing their hashes.

For an ideal hash function, each output bit has approximately a 50% chance of changing when the input changes. For SHA-256, the expected number of changed bits is approximately

$$256 \times 0.5 = 128\ bits$$

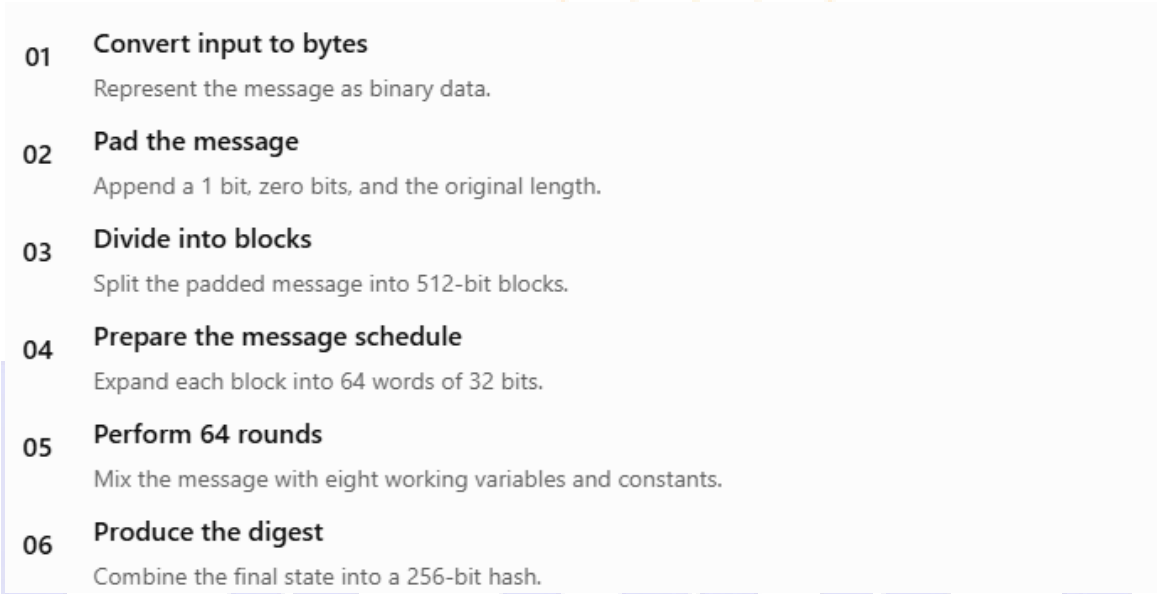
Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

This is an expected statistical property, not a guarantee that exactly 128 bits will change every time.

How SHA-256 works internally

SHA-256 does not simply convert text into a number. It processes the message through a carefully designed sequence of bitwise operations, additions, and 64 rounds of computation for each 512-bit block.

The algorithm has six main stages.



Stage 1: Convert the message into binary

Computers process information as bits: 0 and 1.

For the message abc, using ASCII-compatible UTF-8 encoding:

Character	Decimal value	8-bit binary
a	97	01100001
b	98	01100010
c	99	01100011

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

The complete message is therefore 24 bits long:

01100001 01100010 01100011

3 bytes × 8 bits = 24 bits

At this stage, the message has not been hashed yet. We have only represented it as bytes.

Stage 2: Pad the message

SHA-256 processes data in blocks of 512 bits (64 bytes). But our example contains only 24 bits.

How can a 24-bit message be processed by an algorithm designed around 512-bit blocks?

The answer is padding.

SHA-256 performs the following operations:

1. Append one 1 bit to the original message.
2. Append enough 0 bits so the total length becomes 448 modulo 512.
3. Append the original message length as a 64-bit unsigned integer, in big-endian order.

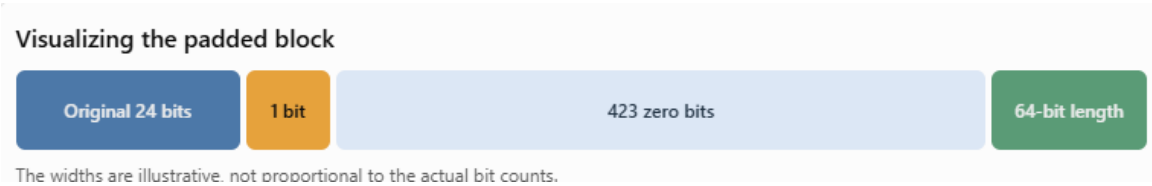
For abc:

- Original message length: 24 bits.
- After appending 1: 25 bits.
- Zero padding: 423 bits.
- Length field: 64 bits.

Therefore,

$$25 + 423 + 64 = 512\ bits$$

The message now fits exactly into one 512-bit block.



Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

The original length is 24, which in 64-bit hexadecimal representation is:

0000000000000018

Why store the original length? It makes the padding unambiguous and is part of the algorithm's prescribed preprocessing.

For longer messages: padding may create multiple blocks. A message that is already close to the end of a block may require an additional block to fit the padding and length field.

Stage 3: Divide the message into blocks

The padded message is divided into 512-bit blocks.

Each block is further divided into sixteen 32-bit words:

$$512 = 16 \times 32$$

These initial words are named:

$$W_0, W_1, \dots\dots\dots\dots\dots\dots\dots\dots, W_{63}$$

The first 16 words come directly from the block. The remaining 48 words are calculated using earlier words.

For t=16,……, 63

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$$

All additions are performed modulo 2^{32}.

The function σ_0 and σ_1 combine rotations, shifts, and XOR operations:

$$\sigma_0(x) = ROTR^7(x) \oplus ROTR^{18}(x) \oplus SHR^3(x)$$

$$\sigma_1(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x)$$

Here:

- `ROTR` means rotate bits to the right.
- `SHR` means shift bits to the right and fill the vacated positions with zeros.
- $\oplus$ means bitwise XOR.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

- For example, rotating the 8-bit pattern `10010110` right by two positions gives `10100101`. Bits leaving the right side re-enter on the left.
- This example uses eight bits just for illustration; SHA-256 operates on 32-bit words.

The schedule helps distribute information from different parts of the original message throughout the 64 rounds, contributing to the avalanche effect.

Stage 5: Perform 64 rounds of processing

This is the core of SHA-256.

The algorithm maintains eight 32-bit working variables:

a,b,c,d,e,f,g,h

These are initialized from the current eight-word hash state. During each of the 64 rounds, the algorithm uses:

- A message-schedule word (W_t).
- A round constant (K_t).
- Bitwise Boolean functions.
- Rotations and modular addition.

The round constants ($K_0, K_1, \ldots \ldots K_{63}$) are fixed 32-bit values defined by the SHA-256 specification.

A. Two important Boolean functions

Choice function — Ch

$$\mathrm{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z)$$

Think of each bit of x as a selector:

- If that bit is 1, select the corresponding bit of y.
- If it is 0, select the corresponding bit of z.

Thus, `Ch` chooses between two inputs bit by bit.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Majority function — Maj

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

For each bit position, the output is 1 when at least two of the three input bits are 1.

For instance:

(x)	(y)	(z)	Ch	Maj
0	0	0	0	0
0	1	1	1	1
1	0	1	1	1
1	1	0	0	1

These functions introduce nonlinear mixing into the computation.

B. The big sigma functions

SHA-256 also uses two rotation-based functions:

$$\sum_0(x) = ROTR^2(x) \oplus ROTR^{13}(x) \oplus SHR^{22}(x)$$

$$\sum_1(x) = ROTR^6(x) \oplus ROTR^{11}(x) \oplus SHR^{25}(x)$$

C. Calculate two temporary values

In each round, SHA-256 calculates:

$$T_1 = h + \sum_1(e) + Ch(e, f, g) + K_t + W_t$$

$$T_2 = \sum_1(e) + Maj(a, b, c)$$

Every addition is modulo 2^{32}.

The working variables are then updated:

$$h \leftarrow g$$

$$g \leftarrow f$$

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

$$f \leftarrow e$$

$$e \leftarrow d + T_1$$

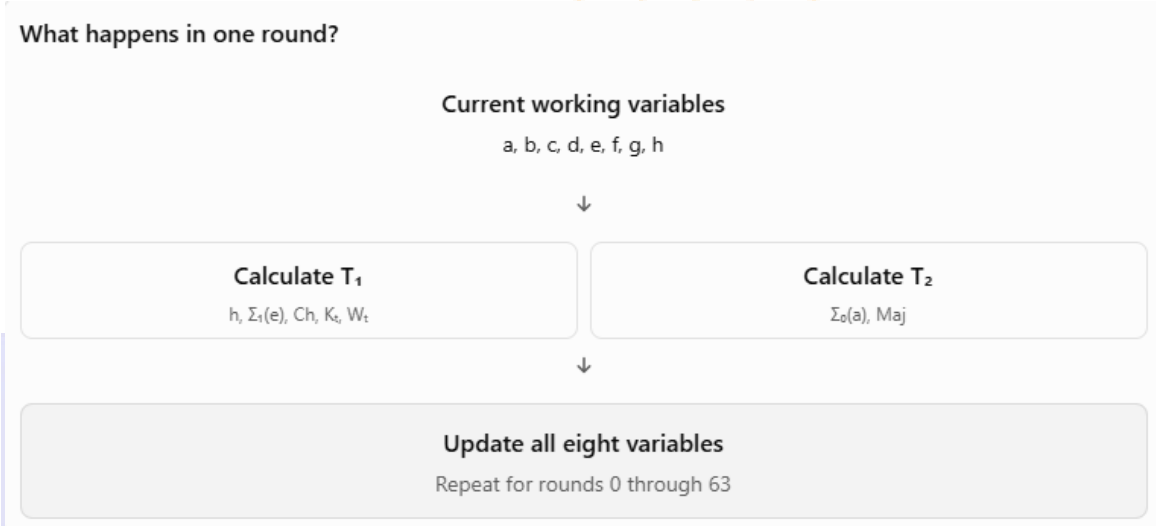
$$d \leftarrow c$$

$$c \leftarrow b$$

$$b \leftarrow a$$

$$a \leftarrow T_1 + T_2$$

These updates happen 64 times for each 512-bit block.



Modular Addition

SHA-256 uses 32-bit arithmetic. If an addition exceeds the maximum 32-bit value, the overflow is discarded:

$$Result = (x + y) mod\ 2^{32}$$

For example

$$(2^{32} - 1 + 1)\ mod\ 2^{32}$$

This is different from ordinary unbounded integer arithmetic.

Stage 6: Produce the final hash

SHA-256 begins with eight predefined 32-bit initial hash values:

$$H_0, H_1, \dots\dots\dots, H_7$$

After the 64 rounds for a block, each working variable is added to its corresponding initial state word, modulo (2^{32}). These updated state values are used for the next block, if one exists.

Once all blocks are processed, the eight final 32-bit words are concatenated:

$$256 = 8 \times 32$$

The result is the final 256-bit digest, usually displayed as 64 hexadecimal characters.

The initial values and round constants are fixed by the SHA-256 specification; they are not randomly generated for each message.

Use Case of SHA 256:

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in



A. Blockchain

Each block can contain the hash of the previous block. If someone modifies the previous block's data, its hash changes, disrupting the chain of hash references. Proof-of-work systems such as Bitcoin also use SHA-256 in their mining process.



B. File integrity

A software publisher can publish a file's SHA-256 hash. After downloading the file, you calculate its hash and compare it with the trusted reference. A match provides strong evidence that the file's bytes are identical, assuming the reference itself is trustworthy.



C. Digital signatures

SHA-256 is often combined with a digital-signature algorithm. The signature verifies authenticity and integrity when checked against the correct public key; SHA-256 alone does not authenticate who created a message.



D. Authentication systems

SHA-256 is used in various cryptographic protocols and constructions. However, using plain SHA-256 to store passwords is not recommended; password-specific algorithms such as Argon2id, scrypt, or appropriately configured PBKDF2 are better choices.

SHA 256 Vs Encryption

This is one of the most important distinctions in cybersecurity.

Property	SHA-256	Encryption
Primary purpose	Create a fixed-length digest	Protect data confidentiality
Key required?	No	Usually yes
Output length	Always 256 bits	Depends on algorithm and input
Recover original input?	Not designed to be reversible	Yes, with the correct decryption key and parameters
Example	Verify a file's integrity	Encrypt a confidential document

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Calculation of Initial Hash Values

The eight initial hash-state words are derived from the fractional parts of the square roots of the first eight primes.

The formula is:

$$\boldsymbol{H_i = [2^{32}\, frac\, (\sqrt{p_i})]}$$

Where:

- p_i is one of the first eight prime numbers: 2, 3, 5, 7, 11, 13, 17, and 19.
- frac(x) means the fractional part of x.
- [x] means round down to the nearest integer.
- Multiplication by 2^{32}converts the fractional part into a 32-bit integer.

For the first prime, p=2:

$$\boldsymbol{H_0 = [2^{32}\, frac\, (\sqrt{2})]}$$

This gives

H_0 = 0x6A09E667

The hexadecimal value is simply another way to represent the resulting 32-bit integer.

Round constants

The 64 round constants are derived from the fractional parts of the cube roots of the first 64 prime numbers:

$$\boldsymbol{K_t = [2^{32}\, frac\, (\sqrt[3]{p_t})]}$$

for the first prime, p=2

$$\boldsymbol{K_0 = [2^{32}\, frac\, (\sqrt[3]{2})]}$$

This gives:

K_0 = 0x428A2F98

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Important distinction: The square-root and cube-root procedures explain how the constants are generated. They do not mathematically prove that SHA-256 is secure. Security depends on the entire algorithm and its resistance to cryptanalysis.

Step 5 — Feed-forward and the final hash

After 64 rounds, the algorithm adds each resulting working variable to the corresponding hash-state word from before the block's compression.

For example:

$$H_0^{new} = \left(H_0^{old} + a\right) mod\ 2^{32}$$

The same procedure is applied to all eight state words.

Each state word contains 32 bits. Concatenating all eight produces:

$$8 \times 32 = 256$$

These 256 bits form the hash state after processing the block. For a message with multiple blocks, the updated state feeds into the next block's processing. After the final block, the eight state words together form the SHA-256 digest.

Numerical Question:

In SHA-256, after completing 64 rounds for a message block, each working variable is added to its corresponding current hash-state word modulo 2^{32}. Consider the following values after the 64 rounds.

Given:

$$2^{32} = 4{,}294{,}967{,}296$$

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Hash-state word	Current hash state	Working variable
H_0	4,294,967,290	15
H_1	100	250
H_2	4,294,967,280	30
H_3	500	200
H_4	1,000	2,000
H_5	4,294,967,295	1
H_6	750	250
H_7	2,000	3,000

Calculate all eight updated hash-state words using modular addition. Then determine the total digest length in bits and the number of hexadecimal characters in the final SHA-256 digest.

Solution

The update rule is:

$$H_i' = (H_i + working\ variable)\ mod\ 2^{32}$$

Step 1:

Update H_0

$$H_0' = (4{,}294{,}967{,}290 + 15)\ mod\ 2^{32}$$

$$= 4{,}294{,}967{,}305\ mod\ 4{,}294{,}967{,}296$$

$$= 9$$

Step 2:

Update H_1

$$H_1' = (100 + 250)\ mod\ 2^{32}$$

$$H_1' = 350$$

Step 3:

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Update H_2

$$H_2' = (4{,}294{,}967{,}280 + 30)\ mod\ 2^{32}$$

$$H_2^{'} = 14$$

Similarly update H_3 to H_7.

Final digest length

SHA-256 concatenates the eight updated 32-bit state words.

$$8 \times 32 = 256\ bits$$

Each hexadecimal character represents 4 bits.

$$\frac{256}{4} = 64\ hexadecimal\ characters$$

Then find corresponding hexadecimal

State word	Decimal value	32-bit hexadecimal word
H_0	9	00000009
H_1	350	0000015e
H_2	14	0000000e
H_3	700	000002bc
H_4	3,000	00000bb8
H_5	0	00000000
H_6	1,000	000003e8
H_7	5,000	00001388

Final 256-bit digest representation

Concatenate all eight hexadecimal words in order, without spaces:

000000090000015e0000000e000002bc00000bb8000000000000003e800001388

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

This is an illustrative digest formed from our example's updated state words. It is not the actual SHA-256 hash of a message.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in